Traveling Salesman Problem (TSP) Using Python

Aim

To find the shortest possible route that visits each city exactly once and returns to the starting city using the Traveling Salesman Problem (TSP) algorithm.

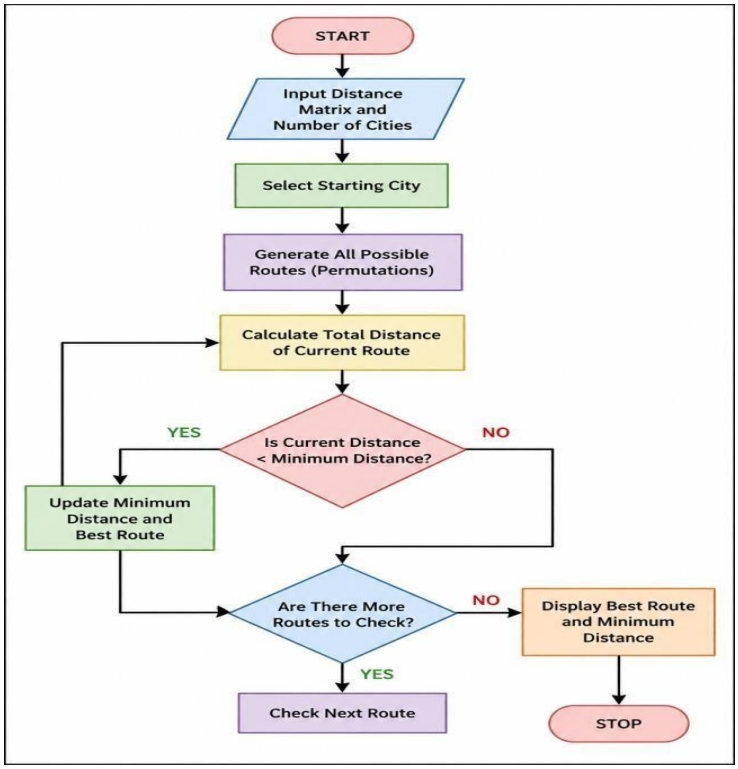
Objective

- To minimize the total travel distance.
- To visit all cities exactly once.
- To return to the starting city after visiting all cities.

Algorithm

1. Input the distance matrix between cities.
2. Select the starting city.
3. Generate all possible routes (permutations).
4. Calculate the total distance for each route. 5. Compare distances and find the minimum.
6. Store the shortest route and its cost.
7. Display the optimal route and minimum distance.

Flowchart



Code: from itertools import

permutations

def tsp(graph, start): cities =

list(range(len(graph)))

cities.remove(start)

min_path = float('inf')

best_route = None

for route in permutations(cities):

current_cost = 0

k = start

for city in route:

current_cost += graph[k][city]

k = city

current_cost += graph[k][start]

if current_cost < min_path:

min_path = current_cost best_route

= (start,) + route + (start,)

return best_route, min_path

graph = [

[0, 10, 15, 20],

[10, 0, 35, 25],

[15, 35, 0, 30],

[20, 25, 30, 0]

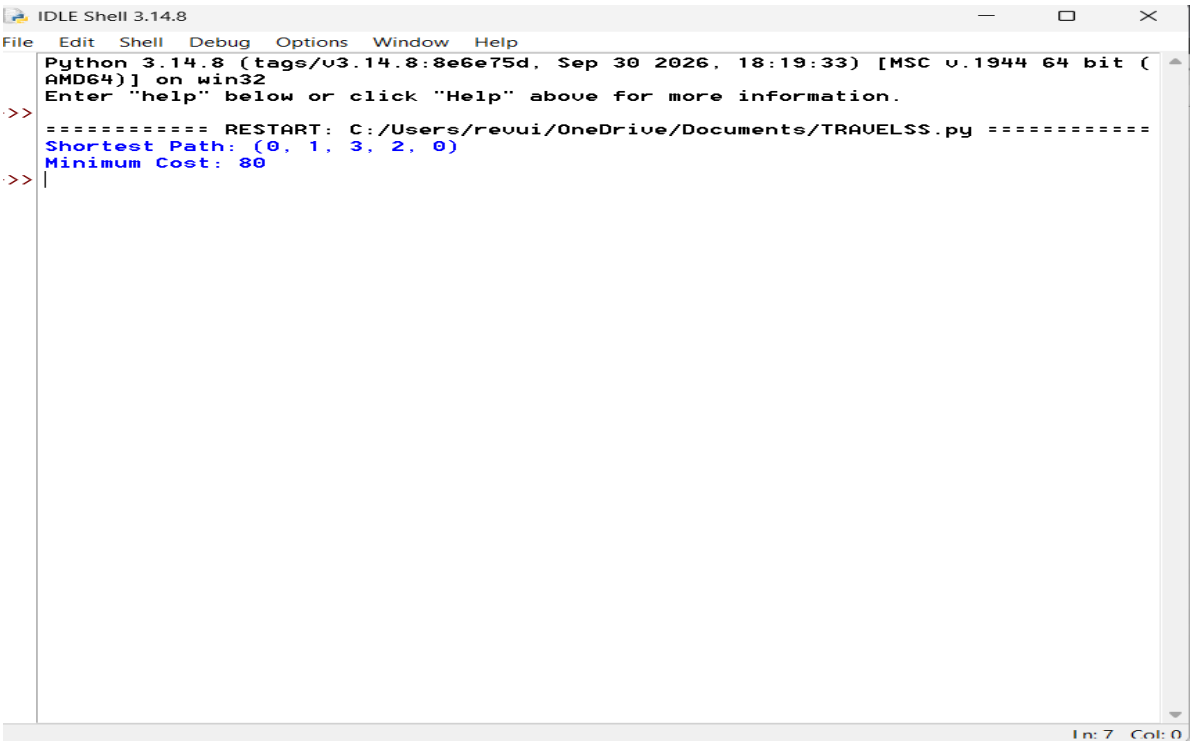
]

route, cost = tsp(graph, 0)

print("Optimal Route:", route) print("Minimum

Cost:", cost)

OUTPUT:



Result

The **Traveling Salesman Problem (TSP)** was solved successfully using Python. The program evaluated all possible routes between the cities and calculated their total travel costs. The optimal route obtained was **0 → 1 → 3 → 2 → 0** with a **minimum cost of 80 units**. Thus, the shortest possible path that visits each city exactly once and returns to the starting city was found successfully.

Conclusion

The Traveling Salesman Problem was successfully implemented in Python. The program evaluates all possible routes and identifies the route with the minimum travel cost. For the given distance matrix, the optimal route is **0 → 1 → 3 → 2 → 0** with a minimum cost of **80 units**.